

STAT 390 - Presentation 3

Team 3

Leah Tan, Xiaolin Liu, Elsa Steen Koppell, Leslie Gonzalez-Flores, and Sofie Langenhuizen

Roles & Responsibilities

Our Team

Leah Consultant

Responsible for suggesting changes and room for improvement in methodology, execution

Sofie Coder

Expert in the code, produces and executes the model, making necessary edits to any redundancies

Xiaolin Coder

Expert in the code, produces and executes the model, making necessary edits to any redundancies

Leslie Mathematician

Expert in the math behind the model, suggest edits to methodology and provide justification for parameters

Elsa Mathematician

Expert in the math behind the model, suggest edits to methodology and provide justification for parameters

Agenda



Objective



Benign Splits



Mandatory Changes



Hyperparameter Tuning



Fixed Variable Image Sizes



Patch Attention Analysis Hierarchy

Agenda

Results



Next Steps



Objective

Modify machine learning model **to more accurately classify** the categories of **C-MIL** (Conjunctival melanocytic intraepithelial lesions)

→ Last time our model was too aggressive → classifying everything as high-grade

Our aim is to **remove aggressive regularization** and allow the model to actually learn as much from the data as possible

→ Hyperparameter Tuning

→ New regularization and attention weight methods

→ Data Augmentation and diversity



Mandatory Changes

- Removed dropout from attention MLP
- Added gated attention and entropy regularization (borrowed from Team 4's presentation 2)
 - Removed `.detach()` method in lines 106 and 122 in [models.py](#) for entropy reg
- Using new splits: `data_splits.npz`

Before attention
analysis fix:

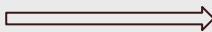
```
Test Results:
Test Loss: 0.3952
Test Acc:  0.8947
Samples:   19
```



```
Test Results:
Test Loss: 0.1626
Test Acc:  0.9474
Samples:   19
```

After attention
analysis fix:

```
Test Results:
Test Loss: 0.7305
Test Acc:  0.7368
Samples:   19
```



```
Test Results:
Test Loss: 0.3590
Test Acc:  0.7368
Samples:   19
```



Why Remove .detach()

→ What .detach() does

- During backpropagation, gradients flow backward from the loss towards input layers, moving in reverse and updating parameters
- .detach() cuts this process off, layers “before” the detach **do not get updated**
 - This prevents regularization from reaching these layering, limiting the positive effects of regularization on accuracy and validation loss
- Removing the detach allows the gradient flow to reach the **attention weights** and **patch embeddings**, i.e. allows regularization to apply to these stages

→ Impact

- Increases efficacy of **weight decay** and **dropout** (regularization works!!)
- Validation loss and accuracy **improve**



Mandatory Changes

- Removed dropout from attention MLP
- Added gated attention and entropy regularization (borrowed from Team 4's presentation 2)
 - Removed `.detach()` method in lines 106 and 122 in [models.py](#) for entropy reg
- Using new splits: `data_splits.npz`

	Base model:		Modified model:		Modified model without removing detach():
Before attention analysis fix:	Test Results: Test Loss: 0.3952 Test Acc: 0.8947 Samples: 19	→	Test Results: Test Loss: 0.1626 Test Acc: 0.9474 Samples: 19		Test Results: Test Loss: 0.3409 Test Acc: 0.8947 Samples: 19
After attention analysis fix:	Test Results: Test Loss: 0.7305 Test Acc: 0.7368 Samples: 19	→	Test Results: Test Loss: 0.3590 Test Acc: 0.7368 Samples: 19		Test Results: Test Loss: 0.3881 Test Acc: 0.7368 Samples: 19



Testing All Benign Splits

Split #	Test Set (7 Cases)	Train/Val Set (28 Cases)
Default (data_splits)	Singletons: 21, 28, 34, 77, 79, 86, 96	Everything else
1	Group B: [26, 109, 110, 111] Singletons: 2, 4, 21	All cases in splits 2, 3, 4, 5
2	Group A: [22, 107, 108] Group C: [24, 118] Group D: [25, 119]	All cases in splits 1, 2, 4, 5
3	Singletons: 23, 27, 28, 30, 34, 36, 43	All cases in splits 1, 2, 4, 5
4	Singletons: 45, 46, 53, 56, 77, 78, 79	All cases in splits 1, 2, 3, 5
5	Singletons: 82, 85, 86, 90, 93, 94, 96	All cases in splits 1, 2, 3, 4



Testing All Benign Splits

Slight caveat (clarified with TA):

- Noticed the cases generated for train, val, and test were the same across all the splits
 - Expected behavior because base sbatch file set the random seed to 9
- Want more randomization → changed the seed for each split (split 1: 9, split 2: 10, split 3: 11, split 4: 12, split 5: 13)

Split 2 and 3:
seed = 9

```
=====
VAL SPLIT
=====
Benign CASE IDs:
[2, 4, 23, 30, 77, 90, 111]
```

```
High-grade CASE IDs:
[17, 18, 19, 49, 50, 63, 69, 70, 83, 84, 97, 104]
```

```
=====
VAL SPLIT
=====
Benign CASE IDs:
[2, 4, 22, 45, 85, 107, 119]
```

```
High-grade CASE IDs:
[17, 18, 19, 49, 50, 63, 69, 70, 83, 84, 97, 104]
```

Split 2: seed = 10
Split 3: seed = 11

```
=====
VAL SPLIT
=====
Benign CASE IDs:
[30, 53, 79, 85, 94, 96, 111]
```

```
High-grade CASE IDs:
[11, 32, 40, 42, 44, 52, 64, 69, 80, 84, 101, 106]
```

```
=====
VAL SPLIT
=====
Benign CASE IDs:
[22, 45, 56, 77, 85, 109, 110]
```

```
High-grade CASE IDs:
[14, 18, 41, 54, 58, 63, 70, 75, 80, 91, 92, 95]
```



Split 1 Results

Test Set (7 Cases)

Group B: [26, 109, 110, 111]
Singletons: 2, 4, 21

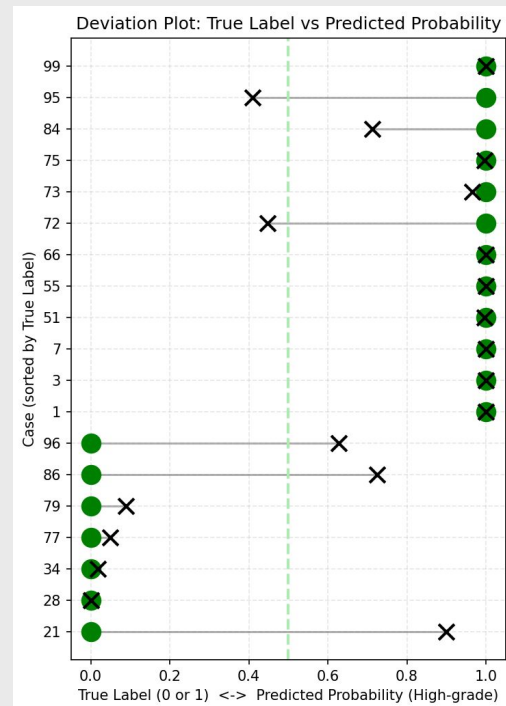
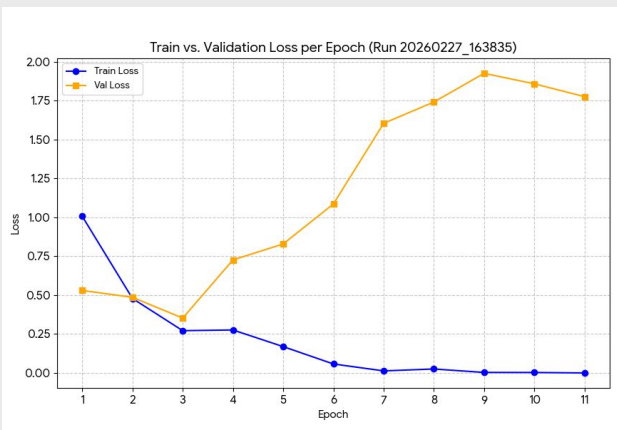
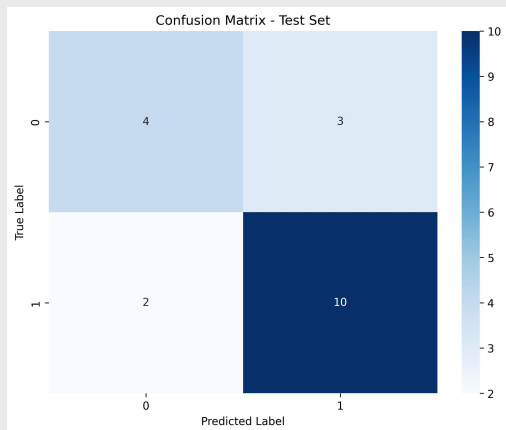
Train/Val Set (28 Cases)

All cases in splits 2, 3, 4, 5

Test Loss: 0.5771

Test Acc: 0.7895

Run time: 0.65 hrs





Split 2 Results

Test Set (7 Cases)

Group A: [22, 107, 108]
Group C: [24, 118]
Group D: [25, 119]

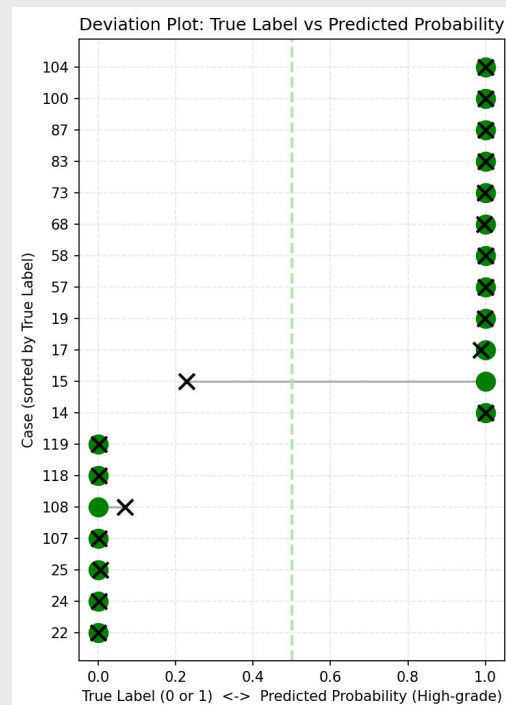
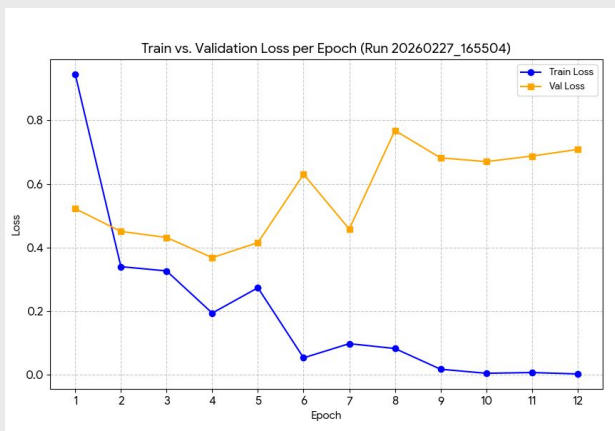
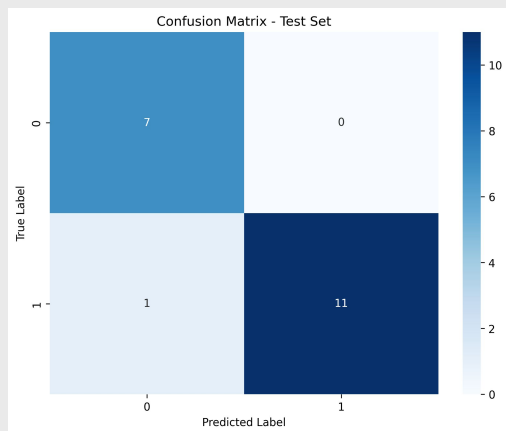
Train/Val Set (28 Cases)

All cases in splits 1, 2, 4, 5

Test Loss: 0.0830

Test Acc: 0.9474

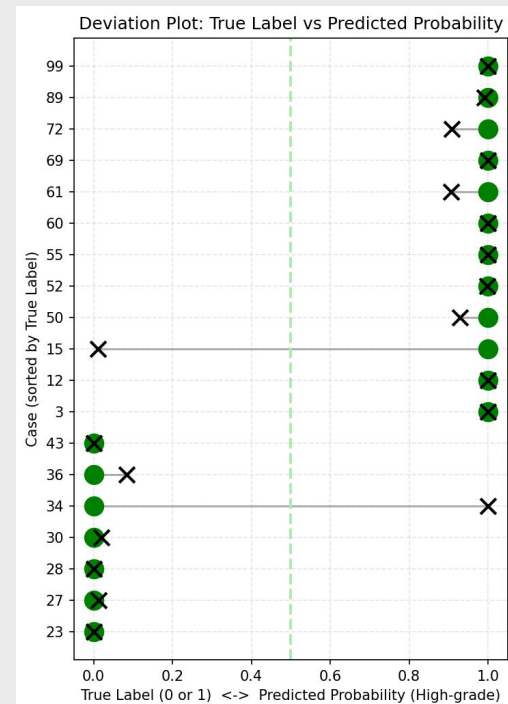
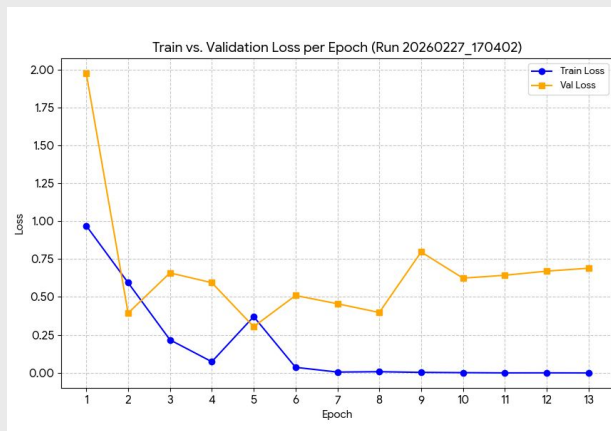
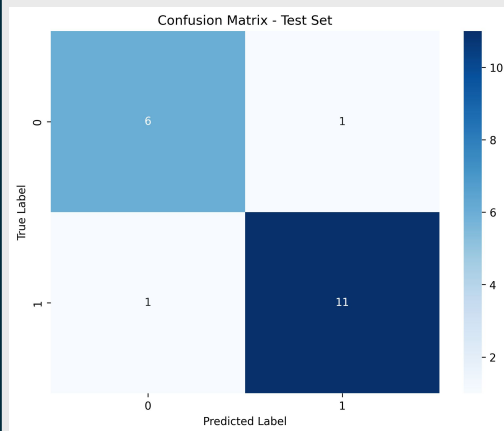
Run time: 0.44 hrs





Split 3 Results

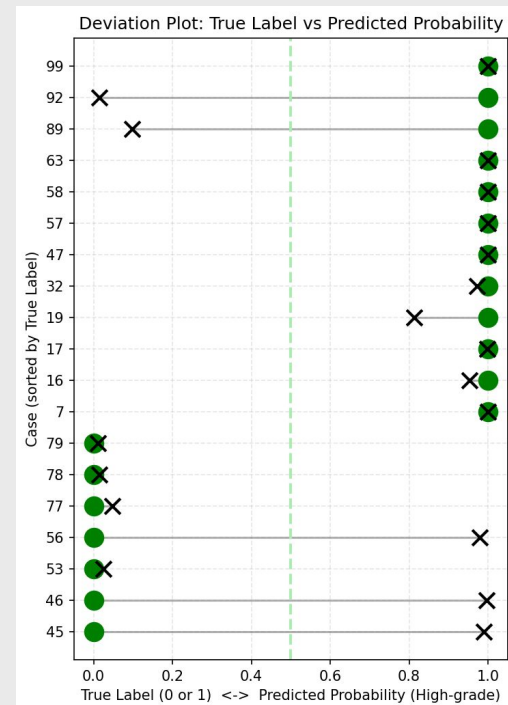
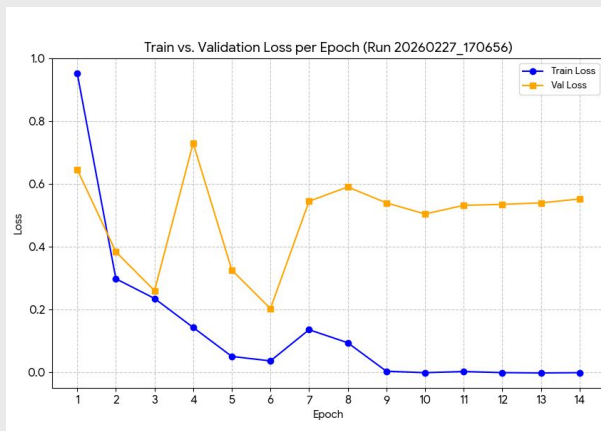
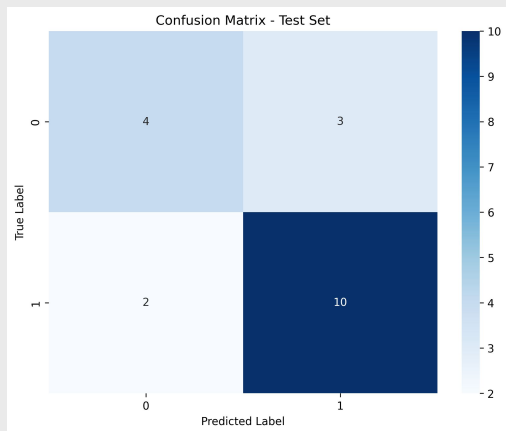
Test Set (7 Cases)	Train/Val Set (28 Cases)	Test Loss: 0.6270 Test Acc: 0.8947 Run time: 0.48 hrs
Singletons: 23, 27, 28, 30, 34, 36, 43	All cases in splits 1, 2, 4, 5	





Split 4 Results

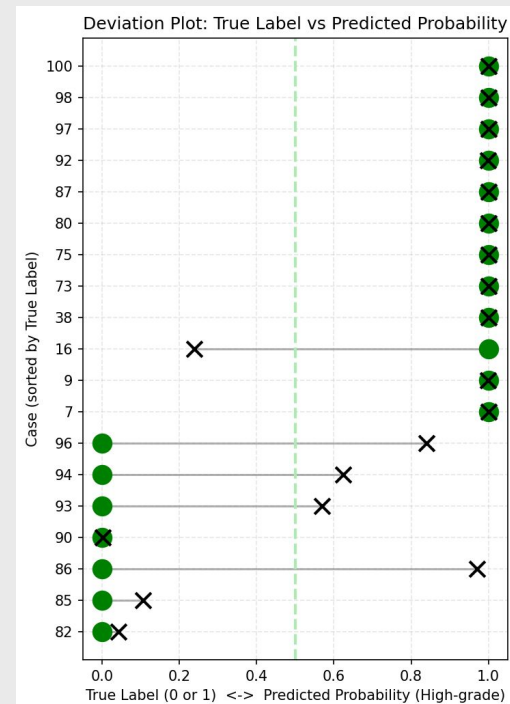
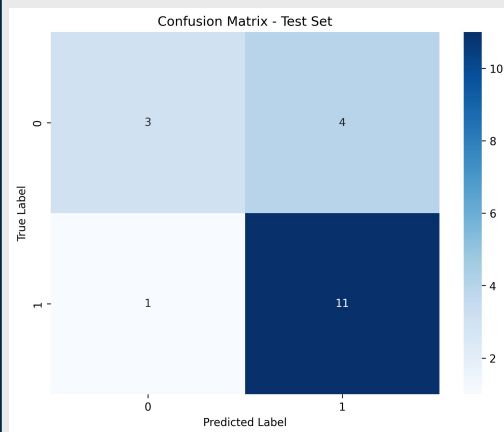
Test Set (7 Cases)	Train/Val Set (28 Cases)	Test Loss: 1.1097 Test Acc: 0.7368 Run time: 1.10 hrs
Singletons: 45, 46, 53, 56, 77, 78, 79	All cases in splits 1, 2, 3, 5	





Split 5 Results

Test Set (7 Cases)	Train/Val Set (28 Cases)	Test Loss: 0.4627
Singletons: 82, 85, 86, 90, 93, 94, 96	All cases in splits 1, 2, 3, 4	Test Acc: 0.7368
		Run time: 0.75 hrs





Some Findings

- Average overall loss: 0.5719
- Average overall accuracy: 0.82
- **Split with best results:** Split 2
 - Test Loss: 0.0830
 - Test Accuracy: 0.9474



- ★ Additional runs
 - Base model using all 5 new splits (without modifications on all the splits) [here](#)
 - Splits 1, 4, 5 had test accuracy of 0.78
 - Splits 2, 3 had test accuracy of 0.94 with split 2 having the best test loss at 0.15



Hyperparameter Tuning

- Bayesian hyperparameter tuning using Optuna
 - Objective: Minimize validation loss
 - Sampler: TPESampler
 - Pruning: MedianPruner, warmup = 6 epochs, startup = 4
- Tuned hyperparameters:
 - Initial Learning Rate: (1e-4, 5e-4)
 - Weight Decay: (5e-5, 5e-4)
 - Patch Projection Dropout: (0.2, 0.5)
 - Classifier Dropout: (0.2, 0.5)
 - Benign Class Weight: (1.5, 3.5)
 - Entropy Lambda: (1e-4, 1e-2)
- Code modified:
 - Tuning script: changed OPTUNA_CONFIG to contain the hyperparameters above, modified the OptunaTrainer with new entropy and dataset changes from base model
 - Models.py, config.py, main.py: broke out single dropout hyperparameter into a separate one for each use case

Tuning script:

/projects/e32998/STAT390_Winter2026/Team3/Code/sofie_presentation3/optuna_training.py

Other scripts modified:

/projects/e32998/STAT390_Winter2026/Team3/Code/sofie_presentation3/models.py, main.py, config.py



Hyperparameter Tuning

- **First 10 trials with 30 epochs and early stopping**
 - 3 trials pruned (Trials 4, 7, 8)
 - 7 remaining trials all stopped early
 - Epoch 10: 1 trial (Trial 5)
 - Epoch 12: 2 trials (Trials 1 & 2)
 - Epoch 13: 3 trials (Trials 0, 3 & 9)
 - Epoch 16: 1 trial (Trial 6)
- **Then 20 trials with 20 epochs and early stopping**
 - 12 trials pruned (Trials 4, 5, 6, 9, 10, 11, 12, 13, 14, 15, 18, 19)
 - 8 remaining trials all stopped early
 - Epoch 10: 1 trial (Trial 0)
 - Epoch 11: 1 trial (Trial 16)
 - Epoch 13: 2 trials (Trials 1 & 7)
 - Epoch 14: 2 trials (Trials 3 & 17)
 - Epoch 15: 1 trial (Trial 8)
 - Epoch 16: 1 trial (Trial 2)

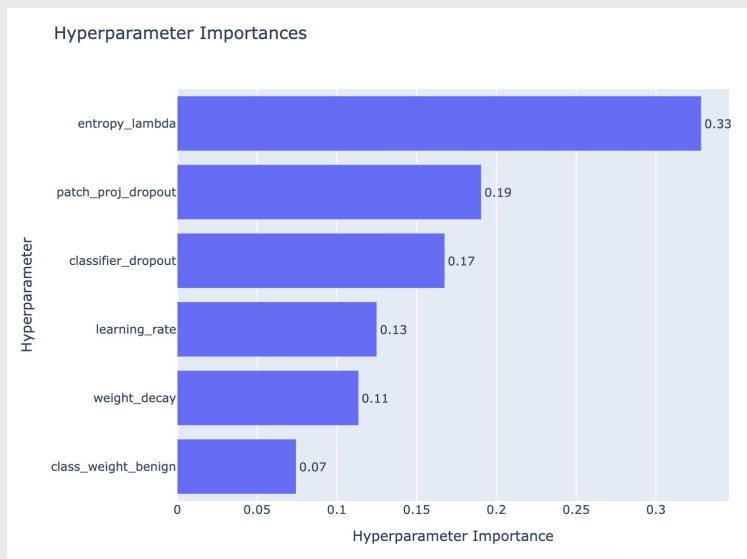
Tuning outputs:

/projects/e32998/STAT390_Winter2026/Team3/Code/sofie_presentation3/output/optuna_tuning_feb26.out, optuna_tuning_feb27.out

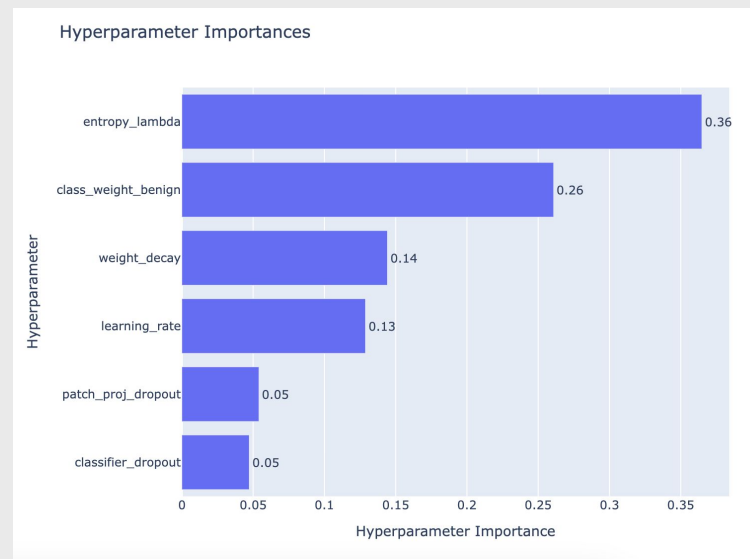


Hyperparameter Tuning

10 trials, 30 epochs



20 trials, 20 epochs



Optuna visualizations:

/projects/e32998/STAT390_Winter2026/Team3/Code/sofie-presentation3/optuna_results_20260226_211353, optuna_results_20260227_165625



Hyperparameter Tuning

Hyperparameter	Base Model	Optimized Model (10 trials, 30 epochs)	Optimized Model (20 trials, 20 epochs)
Initial Learning Rate	2.08e-4	2.62e-4	2.41e-4
Weight Decay	3.05e-4	4.18e-4	7.65e-05
Patch Projection Dropout	0.2599	0.2265	0.4909
Classifier Dropout	0.2599	0.2588	0.4325
Benign Class Weight	2.5285	1.5905	3.3790
Entropy Lambda	1e-3	4.47e-4	6.16e-3

Base model hyperparameters:

/projects/e32998/STAT390_Winter2026/Team3/Code/presentation3_base/config.py

Optimized models best hyperparameters:

/projects/e32998/STAT390_Winter2026/Team3/Code/sofie_presentation3/

optuna_results_20260226_211353, optuna_results_20260227_165625/best_hyperparameters.json



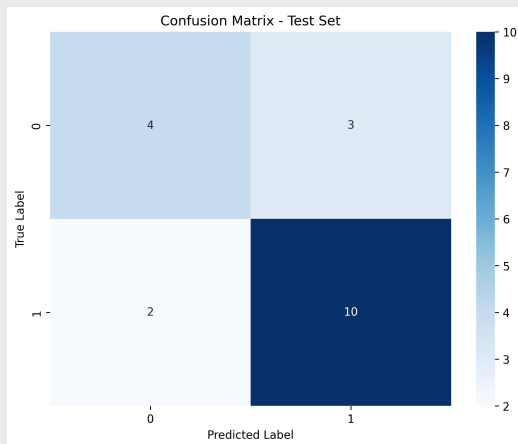
Hyperparameter Tuning

Base Model
(Mandatory changes)

Test Results:

Test Loss: 0.3590

Test Acc: 0.7368

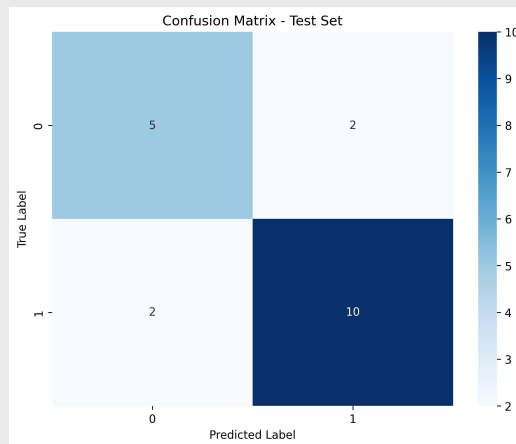


Optimized Model
(10 trials, 30 epochs)

Test Results:

Test Loss: 0.4296

Test Acc: 0.7895

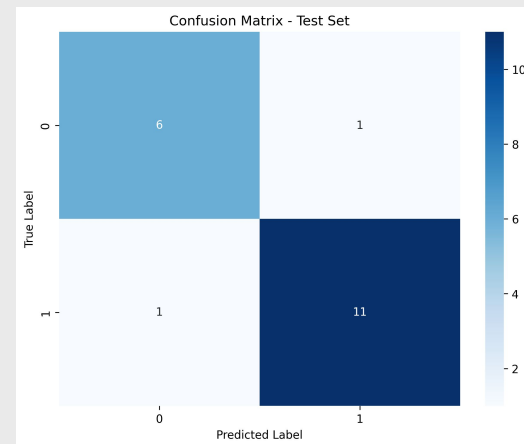


Optimized Model
(20 trials, 20 epochs)

Test Results:

Test Loss: 0.1892

Test Acc: 0.8947



Base model output:

/projects/e32998/STAT390_Winter2026/Team3/Code/sofie_presentation3/output/base_model.out

Optimized model outputs:

/projects/e32998/STAT390_Winter2026/Team3/Code/sofie_presentation3/output/model_best_params_feb26.out, model_best_params_feb27.out



Hyperparameter Tuning

Data Split	Base Model (Mandatory changes)		Optimized Model (20 trials, 20 epochs)	
	Test Loss	Test Acc	Test Loss	Test Acc
Base Split	0.3590	0.7368	0.1892	0.8947
Xiaolin Split 1	0.5545	0.7895	0.4196	0.8421
Xiaolin Split 2	0.2231	0.9474	0.0491	1.0000
Xiaolin Split 3	0.5393	0.7368	0.3875	0.8947
Xiaolin Split 4	1.0304	0.7895	0.4835	0.7895
Xiaolin Split 5	0.6630	0.7895	0.5946	0.8421

Base model outputs:

/projects/e32998/STAT390_Winter2026/Team3/Code/xiaolin_presentation3/output/train_split_1/2/3/4/5.out

Optimized model outputs:

/projects/e32998/STAT390_Winter2026/Team3/Code/sofie_presentation3/output/model_best_params_feb27_xsplit1/2/3/4/5.out



Hyperparameter Tuning

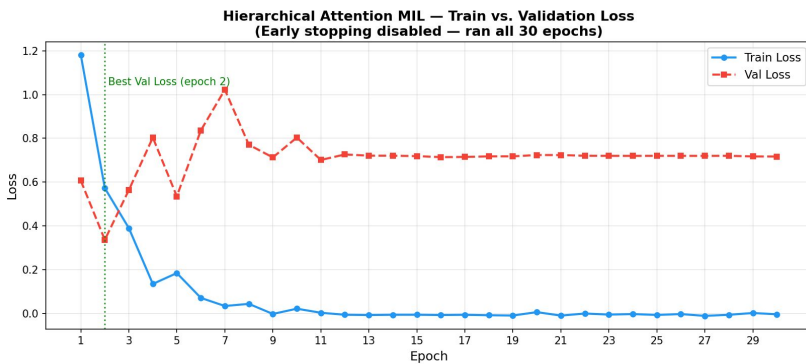
- Retrained our best model, Optimized Model (20 trials, 20 epochs), without early stopping for 30 epochs

Base Split

Test Results:
Test Loss: 0.3148
Test Acc: 0.8947

Xiaolin Split 2

Test Results:
Test Loss: 0.0583
Test Acc: 1.0000

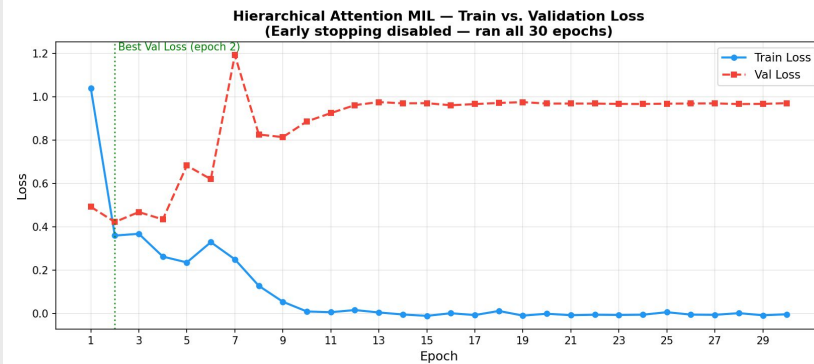


Base split outputs:

/projects/e32998/STAT390_Winter2026/Team3/Code/sofie_presentation3/output/model_best_paramsfeb27_30epochs.out

Xiaolin split outputs:

/projects/e32998/STAT390_Winter2026/Team3/Code/sofie_presentation3/output/model_best_paramsfeb27_30epochs_xsplit2.out





Hyperparameter Tuning

- **Takeaways:**

- Hyperparameter tuning was effective with only 20 epochs, got more effective when increasing from 10 to 20 trials
 - Given that training is much faster now, it would now be feasible to **attempt 30-50 trials with pruning**
- **Entropy lambda** was consistently the most important feature to tune to improve performance with optuna
- **Higher optimized dropout rates** at both the classifier and patch projector level indicate that the randomization introduced with dropout aids in generalization without leading to too much underfitting for our model
- **Higher optimized benign class weight** indicates that we need this stronger aid for dealing with class imbalance
- **Lower optimized weight decay** indicates we may need weaker regularization
- Ideal **initial learning rate** stayed pretty consistent across base and optimized models
 - Likely do not need to tune this in the future given that we are already using a scheduler



Hyperparameter Tuning

- **Optuna Bayesian Optimization Recap:** Finds best hyperparameter configuration by modeling of **good** vs **bad** hyperparameters and sampling from promising regions
- **Gamma / threshold:** threshold for what % of trials are considered “good”
- $D := \{(X_n, y_n)\}$ for $n=1, \dots, N$; , a set of observations (with size $N := |D|$),
 - $D^{(l)}$ = better group, and $D^{(g)}$ = worse groups in D
- Density Functions for better and worse group are created from KDE's *density functions are used to approximate next values of hyperparameters
- Currently, D is initialized through purely random samples. Issue with random sampling cluster of hyperparameter configurations can be created, not allowing for well dispersed set for the TPE algorithm to sample from.
- Potential Sampling Designs: **Sobol low-discrepancy algorithm**



Hyperparameter Tuning

- **Sobol low-discrepancy algorithm**- method of choosing points to fill a dimensional space uniformly with each new point being chosen to fill in gaps that are left and created by the previous points.

Steps using Optuna Sobol* Sampler (relies on SciPy implementation of sobol)

- **Initialization:**
 - Choose number of **dimensions**, i , (in our case 6 for each hyperparameter being tuned)
 - Choose number of **trials**, n , (16 used for example purposes)
 - Sobol works best when in 2^n trials to ensure subspace is divided into evenly spaced out points
- **Index number of trials**
 - Number of trials, 16, have to correspond to sobol indexed $n = 0,1,2...15$
- **Converting trial index to binary**
 - For each trial n , convert n to a binary base 2 value representation
 - n becomes a multidimensional sample point (ex: $n = 5 \rightarrow n = (0101)$; has 4 bits, which is 4 digits in binary)
- **Direction numbers initialized**
 - Each dimension, $i = 1...6$, there is a set of vector direction numbers $v_{i,1} v_{i,2}$ loaded at initialization and are precomputed using primitive polynomials over GF(2) from Joe and Kuo. This define how points should move along each dimension for uniform coverage across dimension space
 - $v_{i,1}$: first direction number from dimension i



Hyperparameter Tuning

- **First Sobol point**

$$X_0 = (0, 0, \dots, 0) \in [0, 1]^6$$

- Point has 6 coordinates to coincide with 6 dimensional space of hyperparameters. Each coordinate represents one hyperparameter value.
- **For each trial $n = 1, \dots, 15$**
 - Identify the first zero bit from least significant side (right side)
 - Ex: $n = 5 \rightarrow n = (0101)$; first zero bit from right side is in the second position
 - Tells us which direction number $v_{i,j}$ to use for XOR to find next Sobol Point
 - XOR - operation that compares two binary numbers, bit by bit. If between both numbers one bit is the value **1**, operation produces **1**. If both numbers are the same, operation produces **0**
 - **Compute next point for each dimension $i = 1, \dots, 6$**
 - Each dimension has its own direction numbers can use, and each dimension is computed independently
 - **Operation**

$$X_{n,i}^{\text{int}} = X_{n-1,i}^{\text{int}} \oplus V_{i,j}^{\text{int}}$$

i = dimension index

1st term = integer value of the coordinate for dimension i in the n th point

2nd term = prev. Point integer value of coordinate for dimension i

3rd term = direction number for dimension i and bit j



Hypertuning Math

- **Normalize Values**
 - Value in previous step is normalized to $[0,1]$ then mapped to hyperparameter value for set ranges
- Procedure is repeated for each trial, and each dimensions (but each dimension has its own set of direction numbers and conducted independently in each trial).
- **Ending number:** 16 Sobol points (hyperparameter configurations) with 6 coordinates (for each dimension)
 - Each coordinate, corresponds to one specific subinterval along that dimension. This is to prevent clustering.
- Algorithm predicts next value by looking at what the previous value has been
- **Number of initial design points proposed: 16**
- **Total number of trials: 64**
- **Reasoning:**
 - Initial design points and total number of trials fit 2^m which ensures even sampling across all dimensions
 - 64 allows for approximately 10 trials to find optimal value of dimension ($10 \times d$)
 - 16 is approximately 25% of total trials. The more trials, the more coverage and sampling of value
 - Budget: 10 Trials with 30 epochs takes approximately 17.5 hours to complete. 16 trials with same number of epochs would take a bit more time. Remaining 48 trials would take approximately 87.5 hours, which is approximately 3.6 days



Hypertuning Math

- **Pruning**
 - **Goal:** stops unpromising training trials early to save computational resources and time. Does this through stopping hyperparameter configurations that show poor intermediate performance
 - **When to prune:**
 - Pruning should occur after model has been able to converged. If done prior to convergence, pruning may stop active model learning. This can prevent potentially high-performing trials from being completed.
 - If model is extremely overfitting, pruning should occur to allow for improved model generalization
 - **Prune too early:**
 - Premature pruning may occur if early validation metrics are misleading
 - This can occur when early epoch show poor validation performance, but later epochs improve
 - If validation metrics fluctuate up and down, pruning may stop a trial before it has fully stabilized
- **Stage Optimization**
 - One search: Train 16 Sobol trials, 48 TPE Trials, but with 30 epochs.
 - Reduced computational expense compared to if ran global search with local refinement
- **Detect Diminishing Returns**
 - Improvement is minimal and less than 1% for multiple iterations (15 epochs). Indicates performance has plateaued and waste of computational resources.



Fixed Image Patch Sizes

→ Computational complexity

- Each patch resized to the same $H \times W$ size
 - Complexity constant across patches

$$O(HWCF)$$

$H \times W$ = chosen fixed image size

C = number of input channels (3 for RGB)

F = number of output feature maps

Because image size (i.e. H & W) is fixed, overall computational complexity is significantly lower than for variable image sizes (see Leslie's slides)

Note: In DenseNet121, the number of output feature maps/channels is 1024

- embed_dim = 512 (post-classifier)



Fixed Image Patch Sizes

→ Ability to detect small pathological regions

- If target patch size is too small, resizing might necessitate downsampling
- Might accidentally cut off (or entirely remove) small pathological regions
 - Signal loss
 - Could potentially avoid this with a good choice of target size
- Higher resolution will also help preserve small details

→ Model accuracy

- Bias: could increase (if resizing eliminates small features)
- Variance: reduced ← input distribution is standardized
- Good choice of target patch size may increase generalizability



Variable Image Patch Sizes

- **Using Team 4 Code Implementation**

- Outer Loop: for each slice (runs S times)
- Patch embeddings list initialization
 - Inner loop: for each patch (runs for number of patches in each slice, P_s)
 - Batch dimension (C,H,W) $\rightarrow$ (1,C,H,W)
 - Feature extraction (variable sizes for h, w , allowed now) (1,1024, h,w)
 - Adaptive pooling (1,1024, 2,2) $\rightarrow$ Flattened (1,4096)
 - Patch embedding from projection (1,512)
 - Concatenate patch embeddings ($P_s * 512$) * P_s is number of patches in that slice
 - Slice embeddings from attention patching embeddings (512)

- **Computational Complexity**

- Since it is for every Slice, and every patch in slice we get the following

$$O\left(\sum_{s=1}^S \sum_{i=1}^{P_s} H_i W_i\right)$$

- Expensive computational cost with double summation, one for each slice, and one for patches in each slice



Variable Image Patch Sizes

- **Ability to detect small pathological regions**
 - Variable patch sizes can allow for small pathological regions to be detected because not fixed to an image size. Therefore, smaller details can be captured and detected without being cut off, or blown up disproportionality that region is distorted.
- **Model Accuracy**
 - **Bias**
 - Variable patch size can reduce bias by having the model learn at different image scales and sizes (fine detailed and broader context images). Variable image sizes can thus prevent underfitting.
 - **Variance**
 - Variable patch size can increase variance. Since each patch is processing one at a time with variable sizes, patch embeddings values can increase in variability. But, in workflow, adaptive pooling can reduce this potential increase in variance by reducing spatial differences to fixed 2×2 .
- **Recommendation**
 - Based on computational complexity cost, and resources, not recommended to introduce variable size. Better to utilize a fixed image size that can balance capturing fine detail and broader image detail. Ideal resolution size can be found through tuning, however this add another dimension to hyperparameter tuning that increases tuning cost in accordance with tuning proposal plan.

→ Patch- vs. Slice-Level Attention

→ Patch-level attention

- What **patches** are most important to each **slice**

$$\mathbf{s}_i = \sum_{p=1}^{P_i} \beta_{p|i} \mathbf{h}_p$$

$\mathbf{s}_i$ = slice i embedding
 $\mathbf{h}_p$ = patch p embeddings
 $\beta_{p|i}$ = attention weights
 P_i = number of patches in slice i

→ Slice-level attention

- What **slices** are most important to each **stain**

$$\mathbf{z}_j = \sum_{i=1}^{Z_j} \alpha_{i|j} \mathbf{s}_i$$

$\mathbf{z}_j$ = stain j embedding
 $\mathbf{s}_i$ = slice i embeddings
 $\alpha_{i|j}$ = slice attention weights
 Z_j = number of slices in stain j

Derived with
`torch.softmax()`

→ Patch-level embedding → slice embedding → stain embedding



Patch- vs. Slice-Level Attention

→ Does a highly peaky patch-attention distribution necessarily imply high slice attention?

- No.
- Peaky patch-attention distribution implies that specific patches *within a slice* are relatively more important for the specific slice
- Even if a few patches dominate one slice, that slice might be relatively low importance *within its stain*

Slice A	
Patch Number	Patch Attention
1	0.89
2	0.04
3	0.02
4	0.01

Slice B	
Patch Number	Patch Attention
1	0.61
2	0.32
3	0.05
4	0.02

Slice C	
Patch Number	Patch Attention
1	0.78
2	0.09
3	0.07
4	0.06



Stain 1	
Slice ID	Slice Attention
A	0.23
B	0.65
C	0.12



Patch- vs. Slice-Level Attention

Peaky post-regularization patch-attention distribution $\Rightarrow$ Strong evidence of class membership

$\rightarrow$ Why?

- Patch and slice attention are locally normalized (sum to 1)
- Regularization focuses on relative attention, not absolute
 - Only compares patches within slice
 - I.e. no cross-slice comparison

Slice A		Slice B	
Raw Patch Attention	Softmax	Raw Patch Attention	Softmax
0.3	0.379	5.0	0.379
0.1	0.310	4.8	0.310
0.1	0.310	4.8	0.310

$$\sum_{p=1}^{P_i} \beta_{p|i} = 1$$

$$\sum_{i=1}^{Z_j} \alpha_{i|j} = 1$$

Despite higher raw attention scores in Slice B, the post-softmax slice embeddings are identical.



Slice-Level Hierarchy (Case 1)

→ Slice attention is approximately uniform across slices

- All slices receive (approximately) equal weight

$$\alpha_{i|j} \approx \frac{1}{Z_j} \implies \sum_{i=1}^{Z_j} \alpha_{i|j} \mathbf{s}_i \approx \frac{1}{Z_j} \sum_{i=1}^{Z_j} \mathbf{s}_i$$

- Effectively, stain embedding is **averaged** slice embeddings
- Without slice-level hierarchy
 - Reduced computation without significant loss of information
 - Perhaps more patch-level variation, result of less normalization
 - Removes redundant-ish structure
- **RECOMMENDATION: Remove slice-level hierarchy.**



Slice-Level Hierarchy (Case 2)

→ Slice attention is highly peaky across slices

- A small number of slices dominate, while the rest are unimportant

For some $\mathbf{k} = \{k_1, k_2, \dots, k_n\} \in \mathbb{Z}^{+2}$, $\alpha_{i|j} \approx 1$ if $i \in \mathbf{k}$ and $\alpha_{i|j} \approx 0$ if $i \notin \mathbf{k}$

- With slice-level hierarchy
 - Filters out noise
 - Irrelevant slices get near-zero weights
 - Most informative slices given more weight
- Without slice-level hierarchy
 - Slice embeddings averaged to get stain embedding
 - Unimportant slices (i.e. low signal) contribute equally (reduced useful structure)
 - Strong signals are diluted (increased signal loss)
- **RECOMMENDATION: Keep slice-level hierarchy.**



Slice-Level Hierarchy (Case 3)

→ Slice attention is uniform in some cases but peaky in others

- Very mixed data, heterogeneity
- Slice attention is adaptive, behaves differently depending on the data

$$\alpha_{i|j} = \begin{cases} \frac{1}{Z_j} & \text{if signal is diffuse} \\ \text{peaky} & \text{if signal is localized} \end{cases}$$

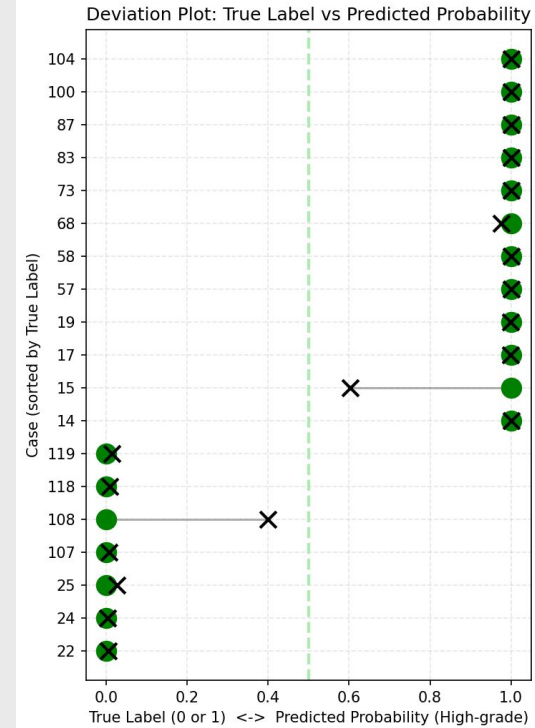
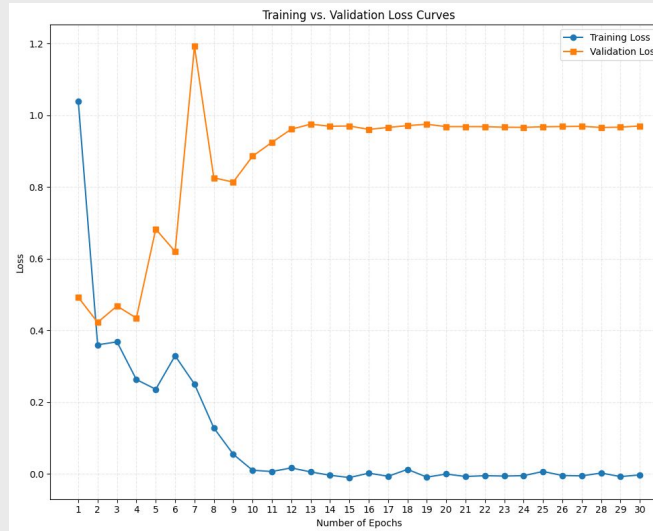
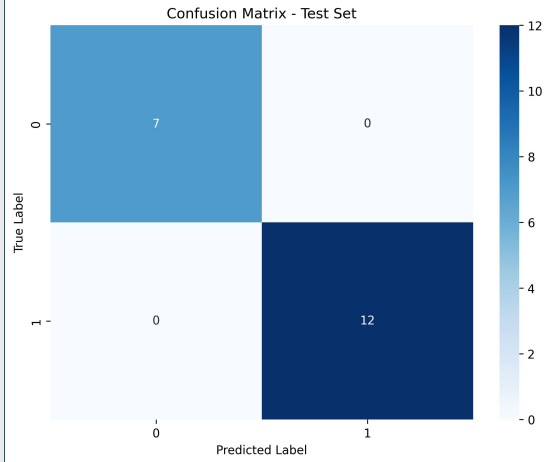
- With slice-level hierarchy
 - Slices treated according to distribution of data (see previous slides for specifics)
 - Provides critical structural information
- Without slice-level hierarchy
 - Slices treated as uniform (Case I), cannot adapt
 - Doesn't have strong effect on cases with diffuse signal
 - Increased signal loss for cases with localized signal
- **RECOMMENDATION: Keep slice-level hierarchy.**



Results

Best model is our hyperparameter tuned model following mandatory changes (on split 2)

Test Accuracy: 100%, Test Loss: 0.0583



Confusion Matrix:

/projects/e32998/MIL_training/final_runs/run_20260228_183748/confusion_matrix.png

Training vs. Val Loss & Deviation Plot Output:

/projects/e32998/STAT390_Winter2026/Team3/Code/leah_presentation3/visualize_predictions.ipynb



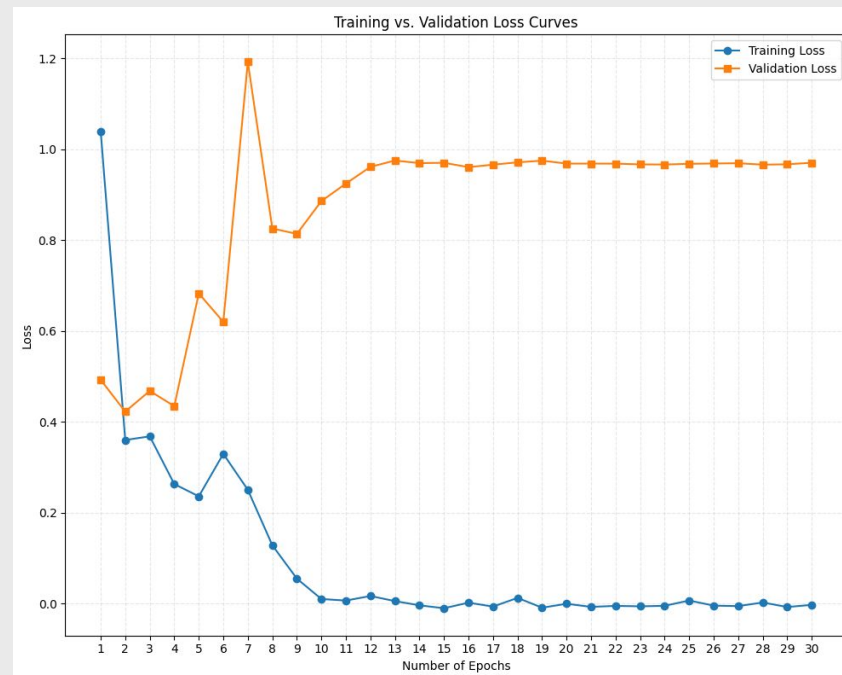
Results

Key Callout:

- Model performs **best around epoch 2 and 3** before it becomes significantly worse performing
- Training **loss reaches "negative" magnitude**
 - ◆ Likely **caused by subtracted "entropy" loss term** to encourage more uniform attention

This suggests that the **model essentially "solves" the training set very quickly** and then the entropy regularization takes over as dominant signal for minimizing loss

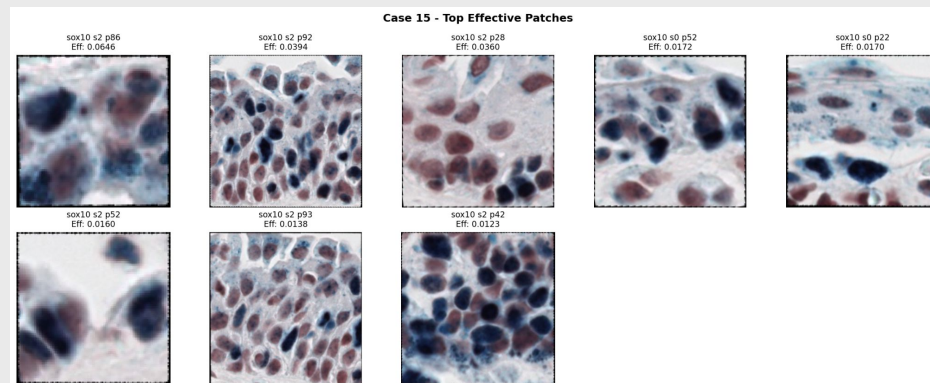
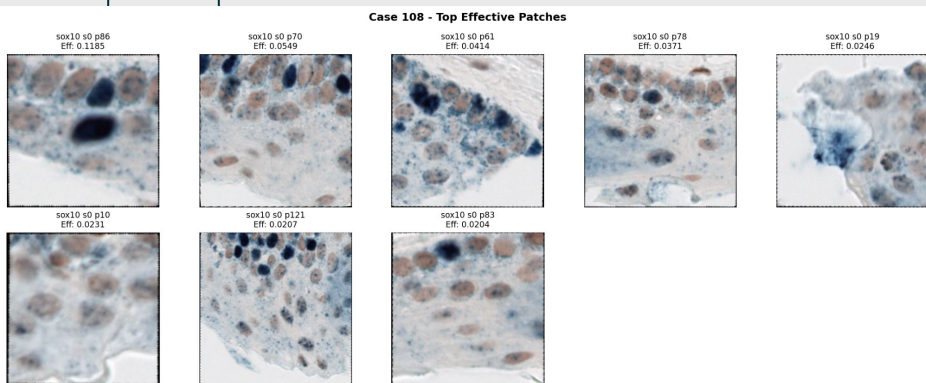
- **Data augmentation** to feed in more data so it doesn't solve it as fast
- **Control entropy regularization** (minimize lambda more, etc.)





Results

No misclassifications, but least accurate of predictions for case **108 (benign, self-made)** and **15 (high-grade)**



Exclusively sox10 stains, while most of the other splits had at least one other stain type within its top effective patch distribution

Additionally, case 108 was only **using patches exclusively from one slice and one stain** → lack of diversity from which model is gathering insights from

Outputs:

[/projects/e32998/MIL_training/final_runs/run_20260228_183748/attention_analysis/case_effective_patches/case_15_top_effective_patches.png](#)
[/projects/e32998/MIL_training/final_runs/run_20260228_183748/attention_analysis/case_effective_patches/case_108_top_effective_patches.png](#)



Results

May be useful for us to further understand not just **what types of patches** are being primarily utilized to gather classification labels, but also **where the patches are relative to the slice itself**

- If misclassified & primarily localized, issue of lack of data diversity or poor image quality
- If misclassified & spread out, could indicate further improvements upon model itself

Building upon this in `/projects/e32998/STAT390_Winter2026/Team3/Code/leah_presentation3/patching_code_leahv2.ipynb`

- Requires all .png masks & .tif slices for tracing
 - ◆ In process of gathering all of it, although inconsistent file structure and naming makes it difficult to match



Takeaways

- Model accuracy improves with splits 1-3, with split 2 as best accuracy overall
 - ◆ Model isn't super generalizable since accuracy varies a lot by split, highlights importance of validating model performance across all splits
- Hyperparameter Tuning sees 20 epochs as more than enough with really early stopping, improved every split
 - ◆ Entropy lambda most important to tune, initial learning rate not very important to tune
 - ◆ Should increase number of trials to further improve performance
- SOBOl as proposed help with more standardized algorithm to create initial design points for hyperparameter configurations
 - ◆ 16 initial trials utilizing SOBOl, switch to TPE with Optuna
- Fixed Image Sizing allows for more computation efficiency without the cost of hurting performance
- Don't remove slice hierarchy entirely if exists any variance within slice attention

Thank you!